

TRABAJO EN CLASE

Taller Funcional

Programación Funcional en Java

Grupo: 4

INTEGRANTES

Jalca Saltos Luiggi

Naranjo Christian

Narváez Lopez Jilson

Materia

Desarrollo de aplicaciones empresariales

Docente

Ing. Christian Merchán Millán

Agosto de 2026

1. Presentación

Este documento presenta el primer entregable del Taller de Programación Funcional en Java. El trabajo desarrolla una aplicación de consola que procesa una lista inmutable de empleados mediante expresiones lambda, interfaces funcionales, Streams, referencias a métodos y Optional.

Resultado del entregable

La clase TallerFuncional.java compila y ejecuta correctamente con JDK 17+. La solución cumple los siete pasos solicitados y añade el reto opcional de agrupación por rango salarial.

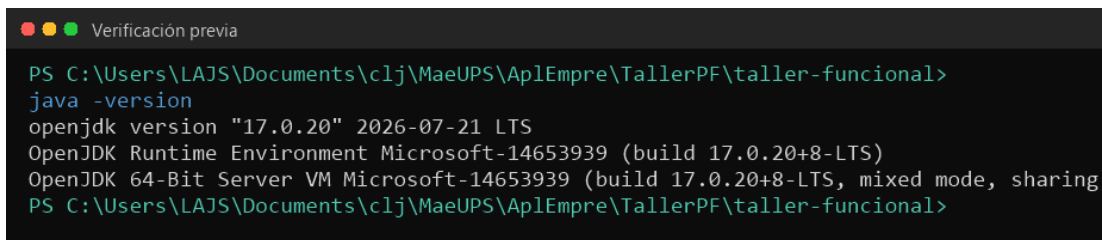
2. Objetivos del taller

- Aplicar los principios de programación funcional en Java a partir de una lista de empleados.
- Filtrar, transformar, ordenar y resumir datos utilizando la API Stream.
- Usar Predicate, Function, referencias a métodos y Optional de forma segura.
- Comprobar que la salida producida coincide con los resultados esperados del taller.

3. Entorno y ejecución

El proyecto se ejecutó con Java 17 o superior. Desde la carpeta del proyecto se emplean los siguientes comandos:

```
cd taller-funcional
javac TallerFuncional.java
java TallerFuncional
```



```
Verificación previa

PS C:\Users\LAJS\Documents\clj\MaeUPS\Ap1Empre\TallerPF\taller-funcional>
java -version
openjdk version "17.0.20" 2026-07-21 LTS
OpenJDK Runtime Environment Microsoft-14653939 (build 17.0.20+8-LTS)
OpenJDK 64-Bit Server VM Microsoft-14653939 (build 17.0.20+8-LTS, mixed mode, sharing)
PS C:\Users\LAJS\Documents\clj\MaeUPS\Ap1Empre\TallerPF\taller-funcional>
```

Figura 1. Verificación de la versión de Java instalada.

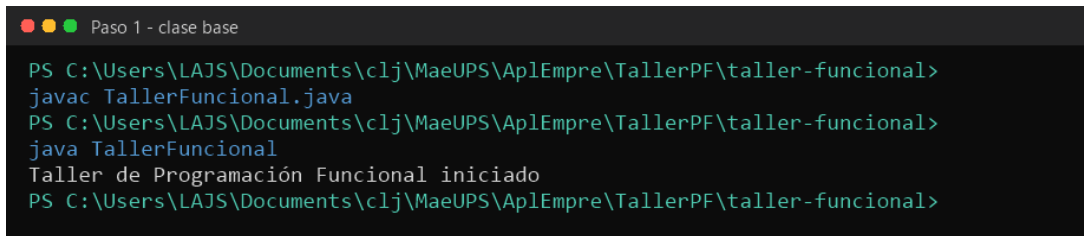
La estructura principal contiene el archivo fuente TallerFuncional.java y una carpeta de capturas que documenta la ejecución de cada paso.

4. Desarrollo del taller

4.1 Paso 1: clase base y comprobación del entorno

Se creó la clase pública TallerFuncional con el método main. Al iniciar, el programa confirma que el entorno está listo mediante un mensaje de consola.

```
public static void main(String[] args) {  
    System.out.println("Taller de Programación Funcional iniciado");  
}
```



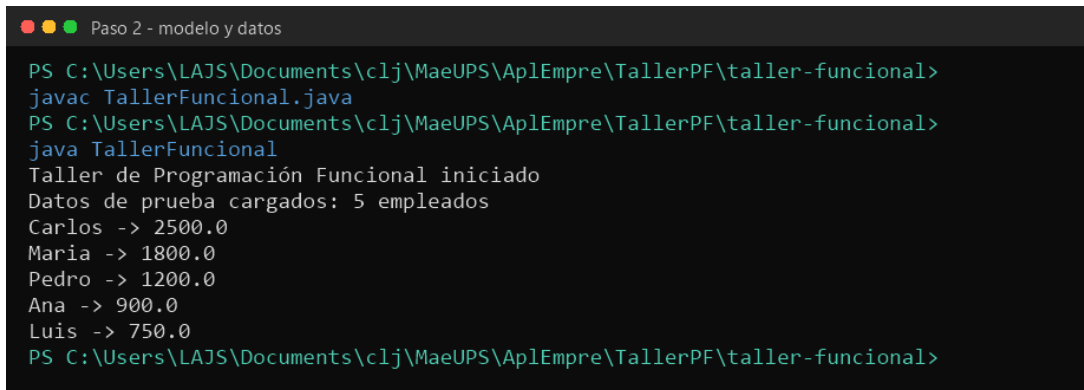
```
Paso 1 - clase base  
PS C:\Users\LAJS\Documents\clj\MaeUPS\AplEmpre\TallerPF\taller-funcional>  
javac TallerFuncional.java  
PS C:\Users\LAJS\Documents\clj\MaeUPS\AplEmpre\TallerPF\taller-funcional>  
java TallerFuncional  
Taller de Programación Funcional iniciado  
PS C:\Users\LAJS\Documents\clj\MaeUPS\AplEmpre\TallerPF\taller-funcional>
```

Figura 2. Creación de la clase base y ejecución inicial.

4.2 Paso 2: modelo Empleado y datos de prueba

Se definió un record inmutable para representar a cada empleado y se cargó una lista inmutable con cinco registros de prueba. Este enfoque evita modificaciones accidentales y se alinea con el paradigma funcional.

```
record Empleado(String nombre, double salario) {}  
  
List<Empleado> empleados = List.of(  
    new Empleado("Carlos", 2500.0), new Empleado("Maria", 1800.0),  
    new Empleado("Pedro", 1200.0), new Empleado("Ana", 900.0),  
    new Empleado("Luis", 750.0)  
);
```



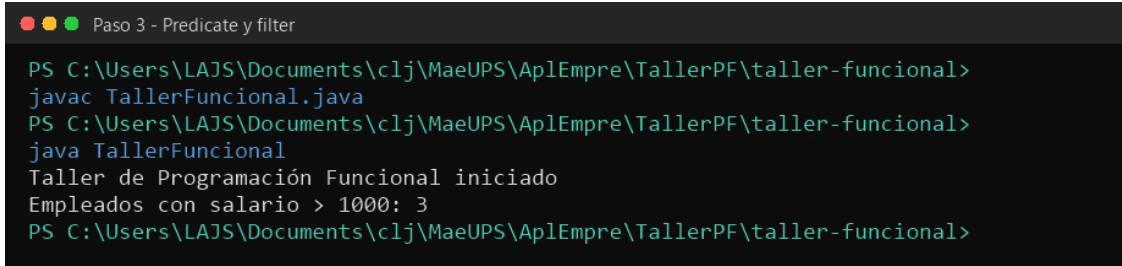
```
Paso 2 - modelo y datos  
PS C:\Users\LAJS\Documents\clj\MaeUPS\AplEmpre\TallerPF\taller-funcional>  
javac TallerFuncional.java  
PS C:\Users\LAJS\Documents\clj\MaeUPS\AplEmpre\TallerPF\taller-funcional>  
java TallerFuncional  
Taller de Programación Funcional iniciado  
Datos de prueba cargados: 5 empleados  
Carlos -> 2500.0  
Maria -> 1800.0  
Pedro -> 1200.0  
Ana -> 900.0  
Luis -> 750.0  
PS C:\Users\LAJS\Documents\clj\MaeUPS\AplEmpre\TallerPF\taller-funcional>
```

Figura 3. Definición del modelo Empleado y lista de datos de prueba.

4.3 Paso 3: filtrado con Predicate y Stream.filter

Se declaró un Predicate que selecciona empleados cuyo salario supera 1000. La operación filter conserva a Carlos, Maria y Pedro, por lo que el resultado contiene tres empleados.

```
Predicate<Empleado> salarioAlto = e -> e.salario() > 1000;
List<Empleado> filtrados = empleados.stream()
    .filter(salarioAlto)
    .collect(Collectors.toList());
```



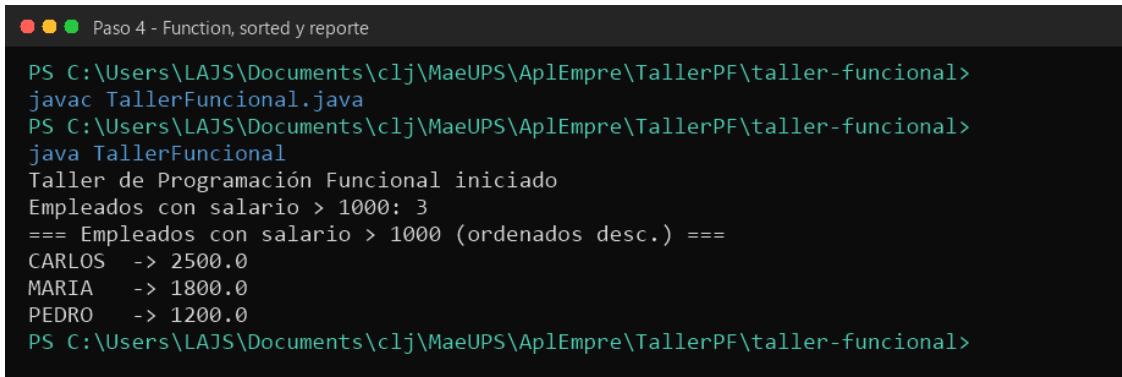
```
PS C:\Users\LAJS\Documents\clj\MaeUPS\AplEmpre\TallerPF\taller-funcional>
javac TallerFuncional.java
PS C:\Users\LAJS\Documents\clj\MaeUPS\AplEmpre\TallerPF\taller-funcional>
java TallerFuncional
Taller de Programación Funcional iniciado
Empleados con salario > 1000: 3
PS C:\Users\LAJS\Documents\clj\MaeUPS\AplEmpre\TallerPF\taller-funcional>
```

Figura 4. Filtrado de empleados con salario mayor a 1000.

4.4 Paso 4: Function, referencias a método y ordenamiento

La Function convierte los nombres a mayúsculas; la referencia Empleado::salario se utiliza al comparar y ordenar de forma descendente. Finalmente, forEach imprime el reporte de los empleados filtrados.

```
Function<Empleado, String> aMayusculas = e -> e.nombre().toUpperCase();
List<Empleado> ordenados = filtrados.stream()
    .sorted(Comparator.comparingDouble(Empleado::salario).reversed())
    .collect(Collectors.toList());
```



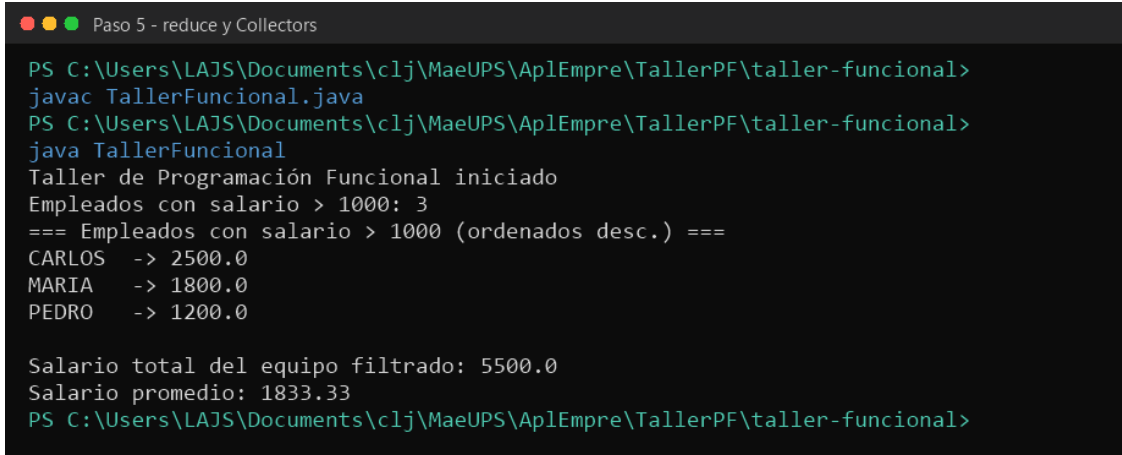
```
PS C:\Users\LAJS\Documents\clj\MaeUPS\AplEmpre\TallerPF\taller-funcional>
javac TallerFuncional.java
PS C:\Users\LAJS\Documents\clj\MaeUPS\AplEmpre\TallerPF\taller-funcional>
java TallerFuncional
Taller de Programación Funcional iniciado
Empleados con salario > 1000: 3
=== Empleados con salario > 1000 (ordenados desc.) ===
CARLOS  -> 2500.0
MARIA   -> 1800.0
PEDRO   -> 1200.0
PS C:\Users\LAJS\Documents\clj\MaeUPS\AplEmpre\TallerPF\taller-funcional>
```

Figura 5. Transformación, referencia a método y ordenamiento por salario.

4.5 Paso 5: totales con reduce y Collectors

Se calculó el salario total mediante map y reduce con Double::sum, y el promedio con Collectors.averagingDouble. Para los empleados filtrados, el total es 5500.0 y el promedio es 1833.33.

```
double total = ordenados.stream().map(Empleado::salario)
    .reduce(0.0, Double::sum);
double promedio = ordenados.stream()
    .collect(Collectors.averagingDouble(Empleado::salario));
```



```
PS C:\Users\LAJS\Documents\clj\MaeUPS\AplEmpre\TallerPF\taller-funcional>
javac TallerFuncional.java
PS C:\Users\LAJS\Documents\clj\MaeUPS\AplEmpre\TallerPF\taller-funcional>
java TallerFuncional
Taller de Programación Funcional iniciado
Empleados con salario > 1000: 3
=== Empleados con salario > 1000 (ordenados desc.) ===
CARLOS -> 2500.0
MARIA -> 1800.0
PEDRO -> 1200.0

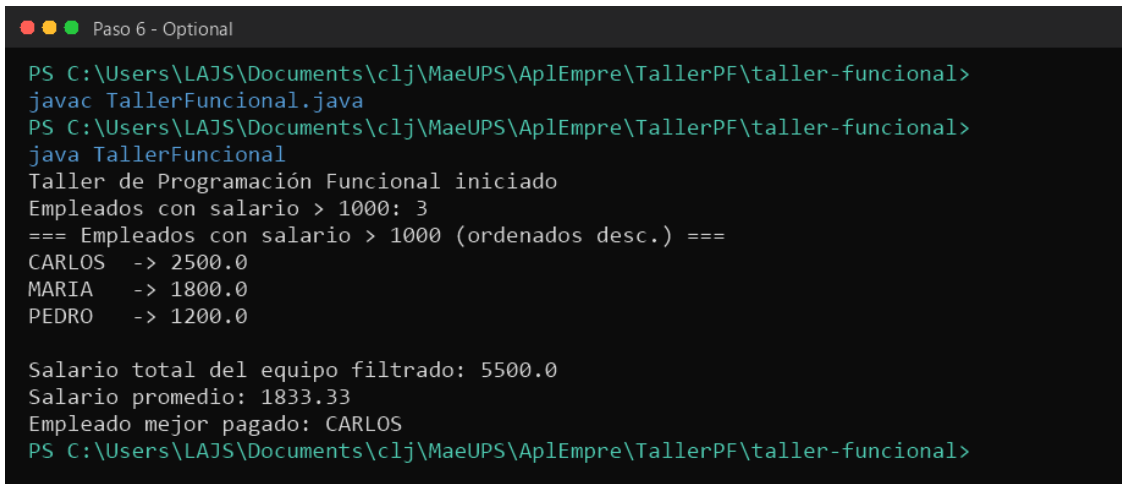
Salario total del equipo filtrado: 5500.0
Salario promedio: 1833.33
PS C:\Users\LAJS\Documents\clj\MaeUPS\AplEmpre\TallerPF\taller-funcional>
```

Figura 6. Cálculo del total y promedio salarial.

4.6 Paso 6: máximo con Optional

El empleado mejor pagado se obtiene mediante Stream.max, cuyo resultado es Optional<Empleado>. La resolución con ifPresentOrElse evita usar get() sin verificar la existencia de un valor.

```
Optional<Empleado> mejorPagado = ordenados.stream()
    .max(Comparator.comparingDouble(Empleado::salario));
mejorPagado.ifPresentOrElse(
    e -> System.out.println(e.nombre().toUpperCase()),
    () -> System.out.println("No hay empleados que cumplan el criterio")
);
```



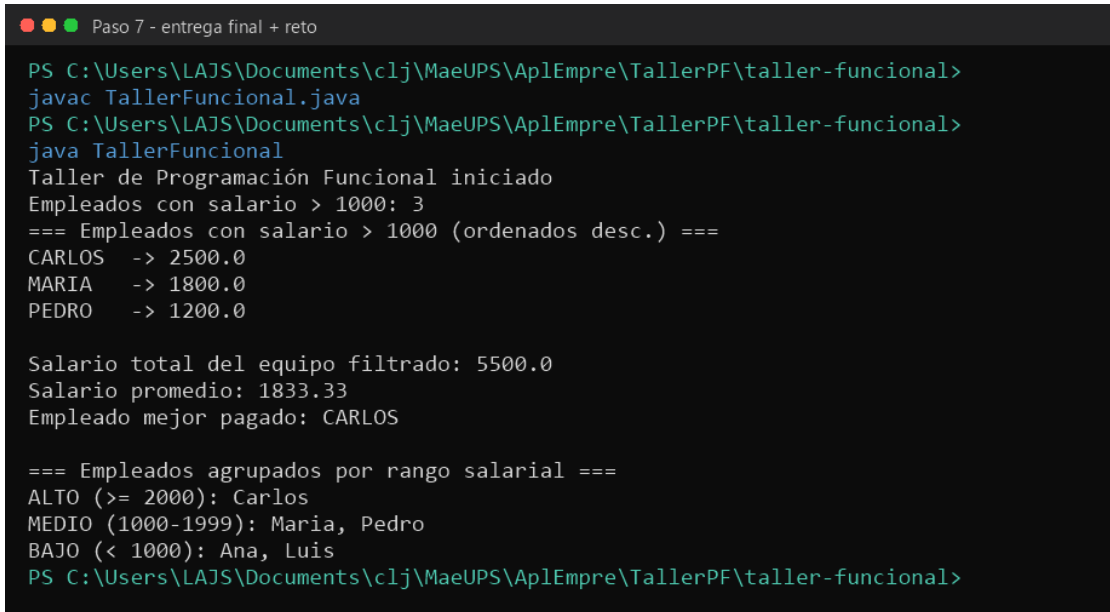
```
PS C:\Users\LAJS\Documents\clj\MaeUPS\AplEmpre\TallerPF\taller-funcional>
javac TallerFuncional.java
PS C:\Users\LAJS\Documents\clj\MaeUPS\AplEmpre\TallerPF\taller-funcional>
java TallerFuncional
Taller de Programación Funcional iniciado
Empleados con salario > 1000: 3
=== Empleados con salario > 1000 (ordenados desc.) ===
CARLOS -> 2500.0
MARIA -> 1800.0
PEDRO -> 1200.0

Salario total del equipo filtrado: 5500.0
Salario promedio: 1833.33
Empleado mejor pagado: CARLOS
PS C:\Users\LAJS\Documents\clj\MaeUPS\AplEmpre\TallerPF\taller-funcional>
```

Figura 7. Obtención segura del empleado mejor pagado con Optional.

5. Resultados y verificación final

La ejecución final confirma el funcionamiento de todos los componentes requeridos. El reporte muestra los empleados ordenados, el total, el promedio y el empleado mejor pagado.



```
PS C:\Users\LAJS\Documents\clj\MaeUPS\AplEmpre\TallerPF\taller-funcional>
javac TallerFuncional.java
PS C:\Users\LAJS\Documents\clj\MaeUPS\AplEmpre\TallerPF\taller-funcional>
java TallerFuncional
Taller de Programación Funcional iniciado
Empleados con salario > 1000: 3
=== Empleados con salario > 1000 (ordenados desc.) ===
CARLOS -> 2500.0
MARIA -> 1800.0
PEDRO -> 1200.0

Salario total del equipo filtrado: 5500.0
Salario promedio: 1833.33
Empleado mejor pagado: CARLOS

=== Empleados agrupados por rango salarial ===
ALTO (>= 2000): Carlos
MEDIO (1000-1999): Maria, Pedro
BAJO (< 1000): Ana, Luis
PS C:\Users\LAJS\Documents\clj\MaeUPS\AplEmpre\TallerPF\taller-funcional>
```

Figura 8. Ejecución final del programa y reto opcional.

Resultados observados

- Empleados con salario superior a 1000: Carlos, Maria y Pedro.
- Orden descendente por salario: CARLOS (2500.0), MARIA (1800.0) y PEDRO (1200.0).
- Salario total del equipo filtrado: 5500.0.
- Salario promedio: 1833.33.
- Empleado mejor pagado: CARLOS.

Reto opcional implementado

Además de lo solicitado, se implementó una agrupación de todos los empleados por rango salarial usando `Collectors.groupingBy` y se generó la lista de nombres de cada rango con `Collectors.joining`. Los rangos mostrados son ALTO, MEDIO y BAJO.

Criterio funcional evidenciado

La solución usa lambdas, referencias a método, Streams y Optional; además, conserva la lista original inmutable y genera nuevos resultados para cada operación.

6. Autoevaluación

¿Qué ventaja ofrece usar Stream frente a un bucle for tradicional en este ejercicio?

Stream permite expresar qué se desea hacer -filtrar, ordenar o sumar- mediante una cadena declarativa. Así se evita mezclar la lógica de acumulación, comparación e impresión en un mismo bucle; el código es más breve, legible y se presta a paralelización si fuese necesaria.

¿Por qué Optional evita un error en tiempo de ejecución al buscar el empleado mejor pagado?

El método `max()` devuelve un `Optional` que puede estar vacío cuando no existen elementos. Con `ifPresentOrElse` se utiliza el empleado solamente cuando está presente y, de no haber coincidencias, se muestra un mensaje alternativo. Esto previene `NullPointerException` y evita llamar a `get()` sin comprobación previa.

¿Qué diferencia existe entre una expresión lambda y una referencia a método?

Una lambda define una función anónima, por ejemplo `e -> e.salario() > 1000`. Una referencia a método, como `Empleado::salario` o `Double::sum`, apunta a un método existente cuya firma coincide con la interfaz funcional requerida; por ello es una forma más concisa de expresar una lambda equivalente.

7. Conclusiones

El taller permitió aplicar de forma práctica las bases de la programación funcional en Java. La solución satisface los resultados esperados al procesar datos sin mutar la colección original y emplear operaciones funcionales para filtrar, transformar, ordenar y resumir la información. El uso de `Optional` fortalece el manejo seguro de resultados potencialmente ausentes.

Anexo A. Código fuente entregado

Archivo: taller-funcional/TallerFuncional.java

```
import java.util.*;
import java.util.function.*;
import java.util.stream.*;

public class TallerFuncional {

    // Java 17+: record immutable, ideal para programación funcional
    record Empleado(String nombre, double salario) {}

    public static void main(String[] args) {
        Locale.setDefault(Locale.US);
        System.out.println("Taller de Programación Funcional iniciado");

        List<Empleado> empleados = List.of(
            new Empleado("Carlos", 2500.0),
            new Empleado("Maria", 1800.0),
            new Empleado("Pedro", 1200.0),
            new Empleado("Ana", 900.0),
            new Empleado("Luis", 750.0)
        );

        Predicate<Empleado> salarioAlto = e -> e.salario() > 1000;
        List<Empleado> filtrados = empleados.stream()
            .filter(salarioAlto)
            .collect(Collectors.toList());
        System.out.println("Empleados con salario > 1000: " + filtrados.size());

        Function<Empleado, String> aMayusculas = e -> e.nombre().toUpperCase();

        List<Empleado> ordenados = filtrados.stream()
            .sorted(Comparator.comparingDouble(Empleado::salario).reversed())
            .collect(Collectors.toList());

        System.out.println("=== Empleados con salario > 1000 (ordenados desc.) ===");
        ordenados.forEach(e ->
```

Anexo A. Código fuente (continuación)

Archivo: taller-funcional/TallerFuncional.java

```
        System.out.printf("%-8s-> %.1f%n",
                           aMayusculas.apply(e), e.salario())
    );

    double total = ordenados.stream()
        .map(Empleado::salario)
        .reduce(0.0, Double::sum);

    double promedio = ordenados.stream()
        .collect(Collectors.averagingDouble(Empleado::salario));

    System.out.println();
    System.out.println("Salario total del equipo filtrado: " + total);
    System.out.printf("Salario promedio: %.2f%n", promedio);

    Optional<Empleado> mejorPagado = ordenados.stream()
        .max(Comparator.comparingDouble(Empleado::salario));

    mejorPagado.ifPresentOrElse(
        e -> System.out.println("Empleado mejor pagado: " + e.nombre().toUpperCase()),
        () -> System.out.println("No hay empleados que cumplan el criterio")
    );

    // --- Reto opcional: agrupar TODOS los empleados por rango salarial ---
    System.out.println();
    System.out.println("=== Empleados agrupados por rango salarial ===");

    Map<String, List<Empleado>> porRango = empleados.stream()
        .collect(Collectors.groupingBy(e -> {
            if (e.salario() >= 2000) return "ALTO (>= 2000)";
            else if (e.salario() >= 1000) return "MEDIO (1000-1999)";
            else return "BAJO (< 1000)";
        }, LinkedHashMap::new, Collectors.toList()));

    porRango.forEach((rango, lista) -> {
        String nombres = lista.stream()
            .map(Empleado::nombre)
            .collect(Collectors.joining(", "));
        System.out.println(rango + ": " + nombres);
    });
}
```